

Nghiên cứu cấu trúc dữ liệu khả tồn

Trần Lập Kiệt

Data Structure / Persistent Data Structures - Chen Lijie

1 Lời nói đầu

Cấu trúc dữ liệu **khả tồn** là cấu trúc lưu lại mọi phiên bản lịch sử của chính nó, đồng thời dùng phần dữ liệu dùng chung giữa các phiên bản để giảm chi phí thời gian và bộ nhớ. Nhiều cấu trúc dữ liệu quen thuộc đều có thể khả tồn hóa, chẳng hạn cây đoạn, cây cân bằng và danh sách khối.

Tài liệu này trước hết trình bày một số ứng dụng của cây đoạn khả tồn, sau đó giới thiệu danh sách khối khả tồn và cây cân bằng khả tồn.

2 Kiến thức chuẩn bị

Các kiến thức được dùng trong bài gồm:

- cây đoạn;
- bao lồi;
- mảng cây dùng để duy trì tổng tiền tố;
- cây cân bằng;
- danh sách khối.

3 Cây đoạn khả tồn

Gọi t là một nút của cây đoạn. Hai con của nó lần lượt là $\text{left}(t)$ và $\text{right}(t)$. Nếu phạm vi của t là $[l, r)$ thì phạm vi của con trái là $[l, (l + r)/2)$, còn phạm vi của con phải là $[(l + r)/2, r)$. Giá trị $f(t)$ lưu thông tin tại nút, ví dụ $\text{cnt}(t)$ là số phần tử trong đoạn đó.

Ta muốn cây đoạn hỗ trợ thao tác cập nhật mà vẫn giữ mọi phiên bản cũ. Trong một lần cập nhật điểm, chỉ có $O(\log n)$ nút trên đường từ gốc đến lá thay đổi. Vì vậy chỉ cần tạo mới các nút này và tái sử dụng mọi nút còn lại.

3.1 Cập nhật điểm

Khi sửa một lá, ta tạo một lá mới chứa giá trị sau khi sửa. Với một nút trong, nhiều nhất một trong hai con bị thay đổi; ta đệ quy xuống con đó, rồi tạo một bản sao mới của nút hiện tại, giữ nguyên con không đổi và trở con còn lại đến phiên bản mới. Do các nút cũ không bị sửa trực tiếp, mọi phiên bản lịch sử vẫn còn nguyên.

3.2 Truy vấn điểm, truy vấn đoạn và đánh dấu

Truy vấn điểm hoặc truy vấn đoạn thực hiện giống cây đoạn thông thường. Với lazy propagation, khi đi qua một nút có dấu, ta đẩy dấu xuống. Việc đẩy dấu không phá vỡ phiên bản cũ vì ta tạo nút mới thay cho các nút con cần thay đổi. Nếu dấu phủ kín một nút, chỉ cần tạo một nút mới biểu diễn đoạn sau khi đánh dấu; nếu không phủ kín, giữ con không liên quan và thay con bị ảnh hưởng bằng phiên bản mới.

4 Danh sách khối khả tồn

Danh sách khối là một mảng được chia thành $O(\sqrt{n})$ khối nối nhau. Ta duy trì kích thước để tổng độ dài của hai khối kề nhau ít nhất $\sqrt{n}$, còn mỗi khối có độ dài không quá $2\sqrt{n}$.

4.1 Thao tác cơ bản

Để chèn hoặc xóa một phần tử, ta tìm khối chứa nó rồi thao tác trong khối với chi phí $O(\sqrt{n})$. Sau mỗi thao tác, quét các khối: nếu hai khối kề nhau có tổng độ dài nhỏ hơn $\sqrt{n}$ thì gộp chúng; nếu một khối dài hơn $2\sqrt{n}$ thì tách thành hai phần gần bằng nhau. Vì thế các thao tác cơ bản đều có độ phức tạp $O(\sqrt{n})$.

4.2 Khả tồn hóa

Khi sửa một khối, ta tạo một khối mới chứa nội dung sau khi sửa. Thay vì dùng danh sách liên kết để lưu các khối, ta lưu dãy khối bằng một mảng kích thước $O(\sqrt{n})$; sau khi sửa, tạo một mảng mới biểu diễn thứ tự khối mới. Cách này giữ lại các phiên bản cũ của cả khối lẫn dãy khối.

5 Bài toán phần tử lớn thứ k trong đoạn

5.1 Phát biểu và các cách cổ điển

Cho dãy $a_0, a_1, \dots, a_{n-1}$. Mỗi truy vấn hỏi trong đoạn $a_l, a_{l+1}, \dots, a_r$, phần tử lớn thứ k là bao nhiêu. Có hai biến thể: chỉ truy vấn, hoặc vừa truy vấn vừa cho phép sửa một vị trí.

Khi không có sửa đổi, một số cách làm quen thuộc gồm:

- nhị phân đáp án và cây đoạn: $O(n \log n)$ tiền xử lý, $O(\log^3 n)$ mỗi truy vấn, $O(n \log n)$ bộ nhớ;
- nhị phân đáp án và chia khối: $O(n \log n)$ tiền xử lý, $O(\sqrt{n \log n \log n})$ mỗi truy vấn, $O(n)$ bộ nhớ;
- cây phân hoạch: $O(n \log n)$ tiền xử lý, $O(\log n)$ mỗi truy vấn, $O(n \log n)$ bộ nhớ;
- cây đoạn xây theo giá trị: $O(n \log n)$ tiền xử lý, $O(\log^2 n)$ mỗi truy vấn, $O(n \log n)$ bộ nhớ.

Khi có sửa đổi, có thể dùng các biến thể kết hợp cây cân bằng hoặc chia khối, nhưng cây phân hoạch tuy nhanh về hằng số lại phức tạp và khó viết hơn. Tài liệu này dùng cây đoạn khả tồn vì dễ hiểu và thuận tiện trong thi OI.

5.2 Dùng cây đoạn tìm phần tử thứ k

Tìm phần tử lớn thứ k tương đương tìm phần tử nhỏ thứ k . Giả sử giá trị đã được rời rạc hóa vào miền $[0, n)$. Ta xây một cây đoạn trên miền giá trị, mỗi nút lưu số phần tử rơi vào đoạn giá trị đó.

Tại một nút t , nếu $\text{cnt}(\text{left}(t)) > k$ thì đáp án nằm trong con trái; ngược lại đáp án nằm trong con phải với thứ tự $k - \text{cnt}(\text{left}(t))$. Mỗi truy vấn như vậy mất $O(\log n)$. Thêm một số chỉ cần cập nhật điểm trên cây đoạn.

5.3 Phép toán trên cây đoạn trọng số

Với hai cây đoạn trọng số có cùng cấu trúc a và b , ta có thể hiểu $a(+/-)b$ là cây mà giá trị cnt ở mỗi nút bằng tổng hoặc hiệu giá trị tương ứng trong a và b . Với một hằng số c , cây $a \cdot c$ nhân tất cả giá trị cnt của a với c . Các phép này thường không cần dựng cây mới; khi truy vấn chỉ cần giữ các con trỏ đến các cây thành phần.

Tổng quát hơn, xét

$$T = \sum_{i=1}^k a_i \cdot c_i.$$

Truy vấn trên T mất $O(k \log n)$ vì tại mỗi tầng phải cộng thông tin từ k cây đoạn.

5.4 Không có sửa đổi

Gọi at_i là cây đoạn trọng số thu được sau khi thêm $a_0, a_1, \dots, a_{i-1}$. Ta có thể tạo at_i từ at_{i-1} bằng một cập nhật điểm. Tất cả các at_i được lưu trong $O(n \log n)$ thời gian và bộ nhớ.

Để hỏi trong đoạn $a_l, \dots, a_{r-1}$, chỉ cần tìm phần tử thứ k trên cây $at_r - at_l$. Độ phức tạp là

$$O(n \log n) + O(\log n) + O(n \log n).$$

5.5 Có sửa đổi

Khi có sửa đổi, một thay đổi có thể ảnh hưởng đến $O(n)$ cây at_i . Ta dùng mảng cây để duy trì hiệu ứng của các sửa đổi, tương đương duy trì tổng tiền tố của các cây đoạn trọng số. Khi đó mỗi at_i biểu diễn được bằng tổng của $O(\log n)$ cây đoạn trọng số, và $at_r - at_l$ là tổng có hệ số của $O(\log n)$ cây như vậy. Truy vấn mất $O(\log^2 n)$, cập nhật cũng chỉ sửa $O(\log n)$ vị trí của mảng cây, nên chi phí sửa đổi là $O(\log^2 n)$.

6 Bài toán phân tử lớn thứ k trên đường đi trong cây

6.1 Phát biểu

Cho một cây có trọng số trên cạnh. Mỗi truy vấn hỏi trên đường đi $a \rightarrow b$, cạnh có trọng số lớn thứ k là bao nhiêu. Cũng có các biến thể online, offline, hoặc hỗ trợ sửa trọng số cạnh.

6.2 Không có sửa đổi

Trước hết xử lý LCA trong $O(n \log n)$ tiền xử lý và $O(\log n)$ mỗi truy vấn. Gọi at_u là cây đoạn trọng số chứa các cạnh trên đường từ gốc đến đỉnh u . Mọi at_u có thể được lấy từ $at_{\text{parent}(u)}$ bằng một cập nhật điểm, nên lưu được trong $O(n \log n)$.

Với đường đi $u \rightarrow v$, cây trọng số của đường đi là

$$T = at_u + at_v - 2 \cdot at_{\text{LCA}(u,v)}.$$

Tìm phần tử thứ k trên T sẽ cho đáp án. Độ phức tạp là $O(n \log n) + O(\log n)$.

6.3 Có sửa đổi

Đánh số các đỉnh theo thứ tự DFS. Khi sửa trọng số một cạnh, chỉ các đỉnh trong một cây con bị ảnh hưởng; trên thứ tự DFS đó là một đoạn. Ta dùng cây đoạn ngoài theo thứ tự DFS, mỗi nút ngoài lưu một cây đoạn trọng số. Khi đó mỗi at_i là tổng của $O(\log n)$ cây đoạn trọng số. Chỉ cần làm khả tồn cây đoạn ngoài cùng để trả lời được truy vấn lịch sử. Độ phức tạp được nêu là

$$O(n \log^2 n) + O(\log^2 n) + O(\log^2 n).$$

7 Từ offline sang online

Cây đoạn khả tồn có thể biến một số thuật toán offline thành online. Ví dụ, với mỗi đỉnh của cây có một trọng số, mỗi truy vấn hỏi tại một đỉnh u , đỉnh đầu tiên trên đường đi lên có trọng số bằng x là đỉnh nào.

Gọi $a[u][c]$ là đỉnh đầu tiên trên đường đi từ u lên có trọng số c . Khi chuyển từ cha f sang u , mảng $a[f][*]$ chỉ thay đổi một vị trí. Vì vậy có thể dùng cây đoạn khả tồn để lưu mọi mảng $a[u]$ với $O(n \log n)$ thời gian và bộ nhớ, rồi mỗi truy vấn trả lời trong $O(\log n)$. Nếu thay cây đoạn khả tồn bằng danh sách khối khả tồn thì có thể đạt $O(n\sqrt{n})$ xây dựng và $O(1)$ truy vấn.

8 Cây cân bằng khả tồn

8.1 Treap

Treap là cây cân bằng trong đó mỗi nút có một khóa ngẫu nhiên, và khóa của cha luôn nhỏ hơn khóa của con. Có thể xem Treap như cây tìm kiếm được chèn theo thứ tự khóa ngẫu nhiên, nên chiều cao kỳ vọng là $O(\log n)$.

Với nút t , ký hiệu $\text{left}(t)$ và $\text{right}(t)$ là hai con, $\text{size}(t)$ là kích thước cây con, còn $\text{key}(t)$ là khóa ngẫu nhiên.

8.2 Hai thao tác nền tảng

Mọi thao tác trên Treap có thể quy về hai phép:

- $\text{merge}(a, b)$, trong đó mọi phần tử của a nhỏ hơn mọi phần tử của b , trả về một Treap chứa toàn bộ phần tử;
- $\text{split}(a, n)$, tách Treap a thành hai Treap, phần trái chứa n phần tử đầu tiên và phần phải chứa phần còn lại.

Chèn một phần tử được thực hiện bằng cách tách theo vị trí rồi hợp nhất ba phần; xóa một đoạn được thực hiện bằng hai lần tách rồi hợp nhất hai phần còn lại.

8.3 Thực hiện merge và split

Với $\text{merge}(a, b)$, nếu một cây rỗng thì trả về cây còn lại. Nếu $\text{key}(a) < \text{key}(b)$, giữ con trái của a , còn con phải mới là $\text{merge}(\text{right}(a), b)$. Ngược lại, giữ con phải của b , còn con trái mới là $\text{merge}(a, \text{left}(b))$. Để khả tồn hóa, mọi thay đổi đều được thực hiện bằng cách tạo nút mới.

Với $\text{split}(a, n)$, nếu $\text{cnt} = \text{size}(\text{left}(a)) \leq n$, tách con trái:

$$\{l, r\} = \text{split}(\text{left}(a), n),$$

rồi thay con trái của a bằng r và trả $\{l, a\}$. Nếu không, tách con phải:

$$\{l, r\} = \text{split}(\text{right}(a), n - \text{cnt} - 1),$$

rồi thay con phải của a bằng l và trả $\{a, r\}$. Mỗi bước cũng tạo nút mới thay vì sửa nút cũ.

Nhờ đó, lấy một đoạn cần hai lần split; xóa một đoạn cần hai lần split và một lần merge.

9 Ví dụ: trình soạn thảo lớn

Cho một chuỗi và ba thao tác: chèn một đoạn chuỗi, sao chép một đoạn bên trong rồi chèn vào vị trí khác, và hỏi ký tự tại một vị trí. Tổng kích thước vào không quá 1 MB, tổng kích thước chuỗi không quá 512 MB, nhưng bộ nhớ chỉ 100 MB.

Treap khả tồn có thể biểu diễn chuỗi và giúp bộ nhớ chỉ phụ thuộc vào số thao tác. Tuy nhiên nếu mọi nút dùng khóa ngẫu nhiên cố định, việc sao chép lặp lại cùng một đoạn có thể làm Treap mất cân bằng. Cách khắc phục là trong phép merge không dùng khóa ngẫu nhiên tĩnh, mà dùng xác suất: nếu hợp nhất a và b , chọn gốc từ a với xác suất

$$\frac{\text{size}(a)}{\text{size}(a) + \text{size}(b)}.$$

Thực nghiệm cho thấy cách này xử lý được nhiều bộ dữ liệu, dù phần chứng minh kỳ vọng chi tiết không được trình bày trong tài liệu gốc.

10 Bao lồi động hoàn toàn

Cho tập điểm trên mặt phẳng. Bao lồi của tập điểm là đa giác lồi có diện tích nhỏ nhất chứa toàn bộ điểm. Bao lồi trên là bao lồi của các điểm cùng $(0, -\infty)$, còn bao lồi dưới là bao lồi của các điểm cùng $(0, \infty)$. Do hai phần đối xứng, chỉ cần xét bao lồi trên.

Bài toán bao lồi động yêu cầu hỗ trợ chèn điểm, xóa điểm và duy trì bao lồi của toàn bộ tập điểm.

10.1 Chỉ có chèn

Nếu chỉ có chèn, đây là bài toán kinh điển: dùng hai cây cân bằng để duy trì bao lồi trên và bao lồi dưới.

10.2 Xóa và hợp nhất bao lồi

Một lần xóa có thể làm thay đổi $O(n)$ điểm trên bao lồi, nên cách trực tiếp không đủ tốt. Khi cần hợp nhất hai bao lồi trên không giao nhau theo trục x , chỉ cần tìm tiếp tuyến chung ngoài rồi ghép hai đoạn thích hợp. Nếu dùng cây cân bằng khả tồn để lưu mỗi bao lồi, việc hợp nhất có thể làm trong $O(\log n)$ nếu bỏ qua chi phí tìm tiếp tuyến.

Để tìm tiếp tuyến giữa hai bao lồi, xét các vị trí tiếp xúc p trên bao lồi A và q trên bao lồi C . Phân tích hình học cho thấy phần “gợn” bị loại nằm ở một trong vài trường hợp; với trường hợp khó, ta xét tiếp tuyến l_p, l_q , giao điểm s và một đường chia m đi qua vùng lõm. Nếu biểu diễn hai bao lồi bằng hai Treap, mỗi thao tác đều làm p hoặc q đi xuống một con, nên tổng chi phí là $O(\log n)$.

Khi đã hợp nhất được hai bao lồi trong $O(\log n)$, ta dùng một cây cân bằng theo thứ tự hoành độ để lưu các điểm. Mỗi nút duy trì bao lồi trên và dưới của tập điểm trong cây con. Chèn hoặc xóa thực hiện trong cây cân bằng; bao lồi ở gốc là đáp án. Chi phí là $O(\log^2 n)$.

11 Ví dụ: Almost

11.1 Phát biểu

Với dãy độ dài $n > 1$, $a_0, a_1, \dots, a_{n-1}$, giá trị *almost average* của dãy được định nghĩa là

$$\frac{\sum_{i=0}^{n-1} a_i}{n-1}.$$

Mỗi truy vấn hỏi trong dãy con liên tiếp $a_l, \dots, a_r$, giá trị *almost average* lớn nhất là bao nhiêu. Ràng buộc: $n \leq 10^5$, $q \leq 3 \cdot 10^4$.

11.2 Chuyển thành hình học

Đặt

$$sum_i = \sum_{k=0}^i a_k, \quad R_i = (i, sum_i), \quad L_i = (i, sum_{i-1}).$$

Khi đó giá trị *almost average* của đoạn $[l, r]$ là hệ số góc của đường thẳng $L_l \rightarrow R_r$. Bài toán trở thành tìm, với $l < i < j \leq r$, hệ số góc lớn nhất của $L_i \rightarrow R_j$. Các hoành độ của L_i và R_i tăng dần.

11.3 Thuật toán $O(\log^2 n)$

Nếu cần tìm hệ số góc lớn nhất của $L_i \rightarrow R_j$ với $i \in [a, b]$, $j \in [c, d]$, $b < c$, thì đó là hệ số góc của tiếp tuyến chung giữa bao lồi dưới của các L trong $[a, b]$ và bao lồi trên của các R trong $[c, d]$. Tiếp tuyến có thể tìm trong $O(\log n)$.

Ta xây một cây đoạn. Mỗi nút ứng với một đoạn $[l, r]$ lưu:

- đáp án tốt nhất nội bộ $l < i < j < r$;
- bao lồi trên của các R trong đoạn;
- bao lồi dưới của các L trong đoạn.

Thông tin của nút cha được ghép từ hai con và một tiếp tuyến giữa hai bao lồi. Xây cây mất $O(n \log n)$. Với một truy vấn, tách khoảng thành $O(\log n)$ nút cây đoạn, quét và hợp nhất dần để được đáp án trong $O(\log^2 n)$.

11.4 Thuật toán $O(\log n)$

Chia dãy thành các khối kích thước $O(\sqrt{n})$. Với mỗi khối, tiền xử lý bao lồi dưới của các tiền tố L , bao lồi trên của các hậu tố R , và đáp án nội bộ. Với mọi cặp khối i, j , cũng tiền xử lý kết quả giữa khối i và khối j ; giá trị này lấy được từ kết quả của i đến $j-1$ trong $O(\log n)$. Tổng tiền xử lý là $O(n \log n)$.

Một truy vấn không nằm hoàn toàn trong một khối được chia thành ba phần: phần đuôi của khối trái, một số khối giữa, và phần đầu của khối phải. Đáp án nằm trong một phần đã tiền xử lý hoặc giữa hai phần, nên tính bằng các bao lồi và tiếp tuyến. Với truy vấn nằm trong một khối, dùng đệ quy cùng ý tưởng. Nếu $F(n)$ là thời gian xử lý kích thước n , ta có

$$F(n) = \sqrt{n} F(\sqrt{n}) + n \log n.$$

Đặt $F(n) = kn \log n$ thì suy ra $k = 2$, nên $F(n) = O(n \log n)$. Vì vậy mỗi truy vấn đạt $O(\log n)$ sau tiền xử lý.

12 Lời cảm ơn

Tác giả cảm ơn Fan Hạo Cường vì đã phổ biến cấu trúc dữ liệu khả tồn trên blog và trong bài giảng WC, cảm ơn Hoàng Gia Thái vì đề xuất cách dùng cây đoạn khả tồn cho bài toán phần tử lớn thứ k trong đoạn và trên cây, cùng các bạn học đã thảo luận vấn đề này.